Projet portfolio — Cluster Haute Disponibilité Linux

Présentation du projet

Ce projet consiste à mettre en place une architecture de **haute disponibilité** pour un service Web dynamique sous Linux, avec un cluster Corosync/Pacemaker, une répartition de charge HAProxy et une réplication MySQL entre deux nœuds de base de données.[file:3][file:5][file:4]

L'objectif est d'assurer la continuité de service, le basculement automatique des ressources critiques et la conservation des données même en cas de panne d'un nœud.[file:3][file:5][file:4]

Architecture mise en œuvre

L'infrastructure s'appuie sur deux nœuds Web gérés par Corosync et Pacemaker, une IP virtuelle pour le service, et un répartiteur HAProxy pour distribuer les requêtes HTTP entre les serveurs disponibles.[file:3][file:5][file:1]

Pour la base de données, la solution évolue d'une réplication maître-esclave vers une réplication MySQL multi-maître afin d'éviter la perte de données lors du retour en ligne d'un ancien nœud principal.[file:2][file:4]

Commandes principales

1. Installation des paquets de cluster

```
sudo apt update
sudo apt install corosync pacemaker pcs
```

Ces paquets installent Corosync, Pacemaker et l'outil `pcs`, nécessaires à la création et à l'administration du cluster Linux.[file:3]

2. Activation du service pcsd

```
sudo systemctl start pcsd.service
sudo systemctl enable pcsd.service
sudo passwd hacluster
```

Le démon `pcsd` doit être démarré sur chaque nœud, puis le même mot de passe doit être défini pour l'utilisateur `hacluster` afin de permettre l'authentification inter-nœuds.[file:3]

3. Authentification et création du cluster

```
sudo pcs host auth weblab1 weblab2
sudo pcs cluster setup haweblab weblab1 weblab2 --force
sudo pcs cluster start --all
sudo pcs cluster enable --all
```

Ces commandes authentifient les nœuds, génèrent la configuration Corosync, puis démarrent et activent le cluster au boot.[file:3]

4. Vérification et paramètres du cluster

```
sudo corosync-cfgtool -s
sudo pcs status
sudo pcs cluster verify --full
sudo pcs property set stonith-enabled=false
```

Elles permettent de vérifier l'état du cluster et, dans le cadre du TP, de désactiver le STONITH pour lever l'erreur de validation de configuration.[file:3]

5. Création de l'IP virtuelle

```
sudo pcs resource create IPFailover ocf:heartbeat:IPaddr2 ip=10.0.0.100 cidr_netmas
sudo pcs resource update IPFailover op monitor interval=10s timeout=30s op stop tim
sudo pcs resource move IPFailover weblab1
```

Cette ressource Pacemaker permet d'attacher l'adresse IP virtuelle au nœud actif afin que le service reste joignable malgré une panne du serveur principal.[file:5]

6. Création de la ressource Web

```
sudo systemctl disable apache2
sudo pcs resource create serviceWeb systemd:apache2 op monitor interval=60s op star
sudo pcs resource group add servWeblab IPFailover serviceWeb
sudo pcs resource defaults migration-threshold=5
```

Le service Web est intégré au cluster et regroupé avec l'IP virtuelle afin que les deux ressources migrent ensemble sur le même nœud.[file:5]

7. Tests de bascule

```
sudo pcs node standby weblab1
sudo pcs node unstandby weblab1
sudo pcs resource cleanup serviceWeb
```

Ces commandes servent à simuler une panne, vérifier la bascule sur le second nœud et nettoyer l'état d'erreur d'une ressource si nécessaire.[file:5]

8. Installation de HAProxy

```
sudo apt update
sudo apt install haproxy
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
sudo systemctl restart haproxy
sudo systemctl reload haproxy
ss -tpnl | grep haproxy
```

HAProxy est utilisé comme répartiteur de charge HTTP, avec validation de la configuration, redémarrage du service et contrôle de l'écoute sur le port 80.[file:1]

9. Exemple de configuration HAProxy

```
frontend proxypublic
    bind 10.0.0.100:80
    default_backend fermeweb

backend fermeweb
    balance roundrobin
    option httpclose
    option httpchk HEAD / HTTP/1.0
    server web1 10.0.0.101:80 check
    server web2 10.0.0.102:80 check
```

Cette configuration met en place une répartition simple de type round-robin avec contrôle de santé des serveurs Web.[file:1]

10. Persistance de session avec cookie

```
backend fermeweb
    balance roundrobin
    cookie PHPSESSID prefix indirect nocache
    server web1 10.0.0.101:80 cookie SW1 check
    server web2 10.0.0.102:80 cookie SW2 check
```

Cette variante permet de conserver une session utilisateur sur le même frontal Web à l'aide du cookie applicatif `PHPSESSID`.[file:1]

11. Configuration MySQL maître-esclave

Sur le serveur maître, le fichier `/etc/mysql/mariadb.conf.d/50-server.cnf` doit contenir :

```
[mysqld]
bind-address = 0.0.0.0
server-id = 100
log_bin = /var/log/mysql/mysql-bin.log
expire_logs_days = 10
```

```
max_binlog_size = 100M
binlog_do_db = appliFrais
```

Cette configuration active les binlogs et identifie le serveur maître pour permettre la réplication MySQL.[file:2]

Sur le serveur esclave :

```
[mysqld]
server-id = 104
expire_logs_days = 10
max_binlog_size = 100M
master-retry-count = 20
replicate-do-db = appliFrais
```

L'esclave est configuré pour se reconnecter automatiquement au maître et ne répliquer que la base utile.[file:2]

12. Commandes SQL de réplication

Sur le maître :

```
FLUSH TABLES WITH READ LOCK;
SHOW MASTER STATUS;
```

Ces commandes verrouillent temporairement les écritures et permettent de récupérer le fichier binaire et la position nécessaires à la configuration de l'esclave.[file:2]

Sur l'esclave :

```
STOP SLAVE;
CHANGE MASTER TO
  MASTER_HOST='172.16.100.101',
  MASTER_USER='replicateur',
  MASTER_PASSWORD='mdpreplicateur',
  MASTER_LOG_FILE='mysql-bin.000001',
  MASTER_LOG_POS=3921;
START SLAVE;
SHOW SLAVE STATUS\G
```

Elles établissent la relation de réplication et permettent de vérifier que `Slave_IO_Running` et `Slave_SQL_Running` sont à `Yes`.[file:2]

13. Passage en MySQL multi-maître

Pour éviter la perte de données après bascule, chaque serveur doit être configuré comme maître et esclave de l'autre, avec l'option `log-slave-updates` et des paramètres d'auto-incrément distincts.[file:4]

Exemple de paramètres à ajouter sur chaque nœud :

```
log-slave-updates = 1
auto-increment-increment = 2
auto-increment-offset = 1
```

Sur le second nœud, la valeur de `auto-increment-offset` doit être différente, par exemple `2`, afin d'éviter les collisions de clés primaires.[file:4]

14. Intégration de MySQL dans Pacemaker

```
sudo pcs resource create serviceMySQL systemd:mysql
sudo pcs resource clone serviceMySQL
sudo pcs status
```

Le service MySQL est intégré au cluster sous forme de clone anonyme, ce qui permet son exécution simultanée sur les deux nœuds dans le cadre de la réplication multi-maître.[file:4]

Compétences mobilisées

- Installation et administration d'un cluster Linux Corosync/Pacemaker.[file:3]
- Mise en place d'une IP virtuelle et gestion du failover applicatif.[file:5]
- Configuration d'un répartiteur de charge HAProxy avec supervision et persistance de session.[file:1]
- Mise en œuvre d'une réplication MySQL maître-esclave puis multi-maître.[file:2] [file:4]
- Réalisation de tests de bascule, de supervision et de validation de service.[file:5] [file:4]

Résultats obtenus

La solution obtenue permet le basculement automatique des services Web en cas de panne, la continuité d'accès via une IP virtuelle, la distribution de charge HTTP entre plusieurs nœuds et la réplication des données MySQL entre serveurs.[file:5] [file:1] [file:4]

Ce projet illustre des compétences concrètes en administration système, haute disponibilité, réseau, supervision de services et gestion d'infrastructure Linux en environnement de production simulé.[file:3] [file:5] [file:4]